SAECOIN

RELEASE MANAGEMENT PLAN

Document: Release_Management_Plan.md
Project: SAECOIN
Ticker: SAC (Provisional)
Version: 1.0
Status: Draft — Under Review
Date: October 2026

---

1. Purpose

This document defines the release management process for SAECOIN software and network components.

Its purpose is to ensure that every software release is:

- Properly tested.
- Version-controlled.
- Documented.
- Reproducible where practical.
- Reviewed before publication.
- Distributed through trusted channels.
- Traceable to its source code.

No software release should be treated as production-ready merely because it compiles successfully.

2. Scope

This policy applies to all SAECOIN software components:

1. Blockchain Core.
2. Full Node.
3. Consensus Engine.
4. Mining Software.
5. Android Wallet.
6. Windows Wallet.
7. Explorer Backend.
8. Explorer Frontend.
9. Developer Tools.
10. Network and operational utilities.

It applies to development releases, testing releases, testnet releases, and mainnet releases.

3. Versioning Policy

SAECOIN will use Semantic Versioning where applicable.

Format:

"MAJOR.MINOR.PATCH"

Example:

"1.2.3"

Meaning:

- MAJOR: Incompatible changes.
- MINOR: Backward-compatible functionality.
- PATCH: Backward-compatible bug fixes.

Pre-release versions may use labels such as:

- "0.1.0-alpha.1"
- "0.2.0-beta.1"
- "1.0.0-rc.1"

A version number alone does not establish consensus compatibility.

4. Release Categories

4.1 Development Release

Purpose:

- Internal development.
- Early feature testing.
- Experimental functionality.

Characteristics:

- May contain incomplete features.
- May contain breaking changes.
- Must not be used to secure real funds.

4.2 Alpha Release

Purpose:

- Initial functional testing.
- Developer testing.

- Early integration testing.

Characteristics:

- Features may be incomplete.
- Data formats may change.
- Not suitable for production use.

4.3 Beta Release

Purpose:

- Broader testing.
- Testnet participation.
- Compatibility testing.
- Performance evaluation.

Characteristics:

- Major functionality should be implemented.
- Known limitations must be documented.
- Not a guarantee of mainnet readiness.

4.4 Release Candidate

Purpose:

- Final validation before a stable release.

Requirements:

- Critical bugs resolved.
- Required tests passed.
- Release notes prepared.
- Compatibility reviewed.
- Security concerns evaluated.

4.5 Stable Release

Purpose:

- General distribution of a reviewed software version.

A stable software release does not automatically authorize a mainnet launch.

Mainnet authorization requires separate approval under the Mainnet Launch Checklist.

5. Release Branch Strategy

The repository should use a controlled branching model.

Suggested branches:

Branch| Purpose
main| Stable reviewed code
develop| Integration of development work
feature/*| New features
bugfix/*| Non-emergency bug fixes
release/*| Release preparation
hotfix/*| Urgent fixes

Branch protection should be enabled for "main".

Direct unreviewed changes to the stable branch should be prohibited.

6. Release Workflow

Every release should follow this process:

Step 1 — Development Freeze

The release manager identifies the release scope.

Only approved changes should enter the release branch.

Step 2 — Code Review

Reviewers inspect:

- Code quality.
- Consensus-related changes.
- Security-sensitive changes.
- Compatibility.
- Error handling.
- Test coverage.

Consensus changes require additional protocol review.

Step 3 — Automated Testing

Required tests may include:

- Unit tests.
- Integration tests.
- Consensus tests.
- Transaction validation tests.

- Wallet tests.
- P2P networking tests.
- Regression tests.
- Fuzz tests.
- Platform-specific tests.

The exact required test suite depends on the component being released.

Step 4 — Build Verification

Builds should be performed using documented development environments.

The project currently targets Windows 11 development with 16 GB RAM.

Build instructions must identify:

- Required compiler versions.
- Required dependencies.
- Build commands.
- Supported platforms.
- Known limitations.

Step 5 — Security Review

Security-sensitive changes must be reviewed before release.

Critical unresolved security issues block release publication.

Step 6 — Release Notes

Every release must include a release note describing:

- Version number.
- Release date.
- New features.
- Bug fixes.
- Security changes.
- Breaking changes.
- Known issues.
- Upgrade instructions.

Step 7 — Tagging

Approved releases should receive signed Git tags where signing infrastructure is available.

Example:

"v0.1.0-alpha.1"

Tags must point to the exact source revision used for the release.

Step 8 — Artifact Publication

Release artifacts should be published through official project channels.

Each release should include:

- Software version.
- Platform information.
- Build information.
- Cryptographic checksums.
- Signature information where available.
- Installation instructions.
- Verification instructions.

7. Build Reproducibility

SAECOIN should work toward reproducible builds.

A reproducible build means independent builders can follow the documented process and produce equivalent artifacts under the defined reproducibility standard.

The project should:

1. Pin dependency versions.
2. Document compiler versions.
3. Record build configuration.
4. Preserve build scripts.
5. Record source commit identifiers.
6. Publish checksums.
7. Investigate unexpected binary differences.

Reproducibility must be demonstrated through testing rather than assumed.

8. Consensus-Critical Release Rules

Consensus-critical software requires special handling.

Examples include changes to:

- Block validation.
- Transaction validation.
- Proof of Work.
- Difficulty adjustment.
- Block reward.
- Supply calculation.

- UTXO validation.
- Cryptographic verification.
- Chain selection.

Such changes must not be released through an ordinary feature approval process alone.

Before release:

1. Update the relevant protocol specification.
2. Explain the compatibility impact.
3. Review attack scenarios.
4. Add regression tests.
5. Test on an isolated network.
6. Conduct independent review where appropriate.
7. Document activation conditions.

A consensus-changing update may require coordinated network activation.

The activation mechanism has not yet been finalized.

9. Database and Data Migration

Software upgrades may change database formats or stored blockchain data.

Before any potentially destructive migration:

- Identify affected data.
- Document compatibility requirements.
- Provide backup instructions.
- Test migration on copied data.
- Define rollback limitations.
- Never silently delete user blockchain data.

Database migrations must be versioned and tested.

10. Emergency Security Releases

Emergency releases may be necessary when a serious vulnerability is discovered.

Examples:

- Consensus failure.
- Critical wallet vulnerability.
- Remote code execution.
- Private-key exposure.
- Severe network disruption.
- Invalid block acceptance.

Emergency procedure:

1. Assess the vulnerability.
2. Restrict disclosure when necessary to reduce exploitation risk.
3. Prepare and review a fix.
4. Test the fix.
5. Determine whether network coordination is required.
6. Publish security instructions.
7. Notify affected users and node operators.
8. Monitor the network after deployment.
9. Publish a post-incident report when safe.

Emergency releases must still preserve source traceability and release integrity.

11. Release Approval

Release approval should depend on the component and its risk.

Release Type| Required Review
Documentation| Documentation review
UI changes| Code review and UI testing
Wallet changes| Code review and wallet security testing
Mining changes| Mining and performance testing
Node changes| Node integration and regression testing
Consensus changes| Protocol review and consensus testing
Mainnet release| Full launch-readiness approval

The project must identify release maintainers and authorized approvers before public releases begin.

12. Supported Versions

The project should publish a support policy for stable releases.

The policy must identify:

- Currently supported versions.
- Security update availability.
- Upgrade requirements.
- End-of-support dates.
- Compatibility restrictions.

Support periods have not yet been finalized.

13. Rollback Policy

Rollback procedures depend on the type of release.

For ordinary application releases:

- Users may be able to reinstall a previous compatible version.
- Wallet data must be protected.
- Database compatibility must be checked.

For node releases:

- Reverting software does not necessarily reverse blockchain state.
- Database rollback may require restoring a verified backup.
- Operators must follow documented recovery procedures.

For consensus changes:

- A software rollback may cause nodes to follow different chain rules.
- Network coordination may be necessary.
- No automatic rollback should be assumed safe.

14. Release Security

Release infrastructure must be protected.

Recommended controls:

- Multi-factor authentication for maintainers.
- Restricted repository permissions.
- Protected release branches.
- Signed release tags where available.
- Checksums for downloadable artifacts.
- Secure handling of signing keys.
- Dependency vulnerability monitoring.
- Audit logs for release publication.

Private signing keys must never be committed to the repository.

15. Release Documentation

Each release should have a corresponding record.

Suggested directory:

"docs/releases/"

Example:

```
docs/
└── releases/
    ├── v0.1.0-alpha.1.md
```

```
├── v0.2.0-beta.1.md
└── v1.0.0.md
```

Release records should include all information necessary to identify and verify the release.

16. Release Readiness Checklist

Before publishing a release:

- [ ] Version number assigned.
- [ ] Release scope documented.
- [ ] Code review completed.
- [ ] Required tests passed.
- [ ] Security review completed where required.
- [ ] Compatibility checked.
- [ ] Database migration reviewed where applicable.
- [ ] Release notes prepared.
- [ ] Build environment documented.
- [ ] Artifacts generated.
- [ ] Checksums generated.
- [ ] Signatures verified where available.
- [ ] Release approval recorded.
- [ ] Official publication location confirmed.

17. Current Project Limitations

At the time of this document:

- No production release is claimed.
- No mainnet software is claimed to exist.
- Release maintainers have not been formally appointed.
- Release signing infrastructure has not been established.
- Reproducible builds have not been demonstrated.
- Stable version support periods have not been finalized.
- Consensus upgrade activation rules remain undecided.

These items must be resolved before production deployment.

18. Conclusion

This Release Management Plan establishes a controlled approach to building, reviewing, testing, publishing, and maintaining SAECOIN software.

The project must prioritize source traceability, security, compatibility, and testing.

No release should be considered production-ready without satisfying its applicable review and approval requirements.

Document: Release_Management_Plan.md
Project: SAECOIN
Version: 1.0 Draft
Status: Under Review